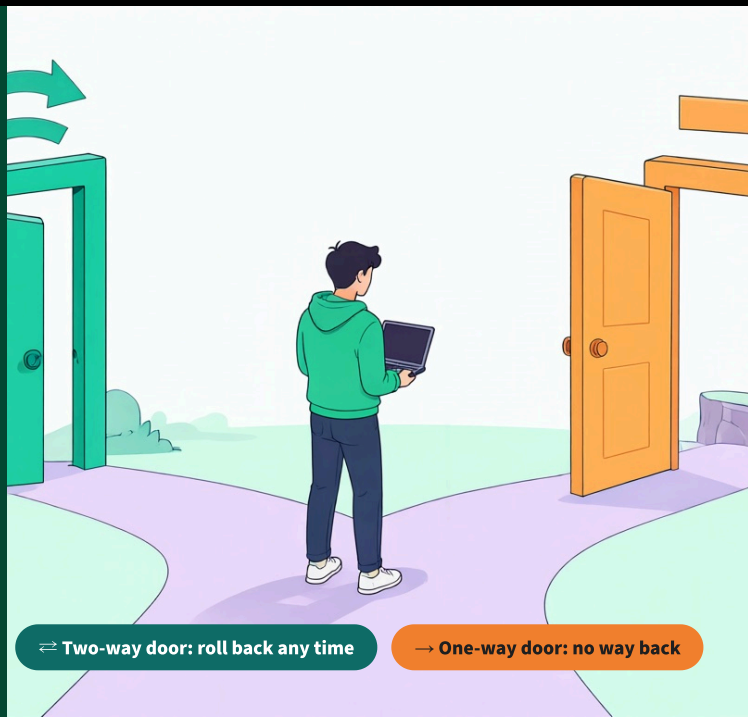
Hackathon proposal · condensed v3

Know before you merge whether you can roll back.

One-Way Door is a GitHub Copilot skill. On every pull request it answers the question no pipeline asks: **if this goes wrong after deploy, can we actually roll back?**



The problem

Rollback is every team's first response to a bad release, yet nobody checks that it works. When a release changes stored data or a message format, the old code may not run against what the release leaves behind. Teams find out mid-incident, and nothing before merge checks for it.

1
sensible refactor: money from a double to whole pence. Tests green, review approved.

50
settlements booked at £0.00 after the rollback. No exception, nothing in the dead-letter topic.

£12,708
missing from the ledger, with every dashboard green. Finance's job broke first.

4
small PRs instead, each one proven to roll back cleanly. Same end state.

The solution, in one folder

- **Classifies every change** in a PR as two-way (safe to roll back) or one-way.
- **For each one-way change:** when the door closes, what breaks on rollback, and the safe rollback window.
- **Finds the blast radius:** other code that reads the same table, column, topic or field, even outside the PR.
- **Proposes a rollback-safe split** (expand → migrate → contract → drop), and generates it on request.

VS Code, Copilot CLI, the cloud agent and code review. Offline: git and Node only.

Proven live, not mocked

An invented payments company, **Northwind**, runs three real Java services, Kafka and Postgres on one laptop. One PR is merged, rolled back, and books £0.00. The skill predicts each failure before merge, and its split passes a five-stage rollback test at every step.

The people in the story



Priya
Developer. Raises PR #482.



Tom
Tech lead. Approves it.



Sam
On call. Presses rollback.



Grace
Finance. Owns the reconciliation.



Dev
DBA. Writes the hotfix.



Hinge
The skill. Asks first.

THE PROBLEM

Rollback is the safety net nobody tests

Code rolls back; state doesn't. A release that changes a table, an event or a stored format leaves data behind, and the previous version has to live with it. Nobody checks, before merge, that it can.

How a rollback goes wrong

Release N ships it changes a schema, an event or a format › **New state is written** tables migrated, events published, caches filled › **An incident** related or not, the plan is to roll back › **Roll back to N−1** the code goes back; the state stays › **Old code, new state** fails, errors, or silently misreads

Four kinds of one-way change

Databases

Change: a column renamed, dropped or retyped; values converted in place.

Old code fails its schema check, or queries a column that's gone.

Events and messages

Change: a field removed, renamed or retyped; its units changed.

Old consumers read the missing field as zero or null, silently.

Caches and state

Change: a new serialisation format, new keys, rewritten idempotency records.

Old code can't read them: misses, errors, or work done twice.

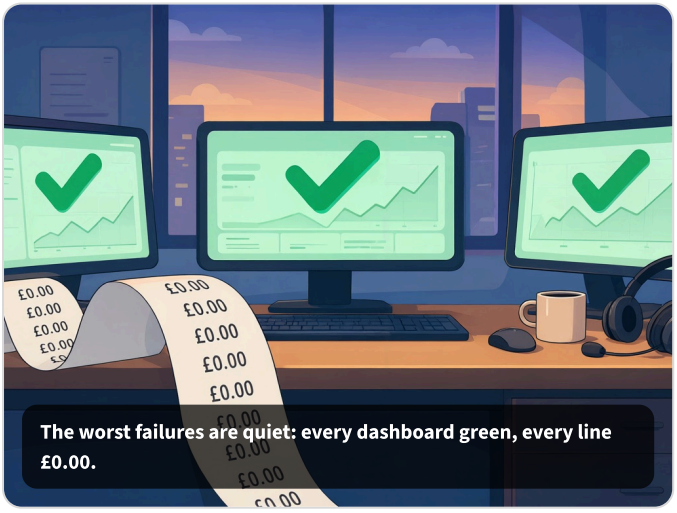
External contracts

Change: webhooks in a new shape, API versions retired, topics deleted.

A redeploy can't take back what partners already received.

Why today's safeguards miss it: they all look forward

Safeguard	What it checks	What it never sees
Unit and integration tests	new code against a fresh database	**old code against data the new code wrote**
CI/CD pipelines	deploying forward	**deploying back after the state has changed**
Migration tools	applying changes in order	**undoing them: most only go forward**
Code review	the lines in the diff	**other services and jobs that read the same data**
Monitoring and alerts	errors and exceptions	**wrong values that raise no error**



The worst failures are quiet: every dashboard green, every line £0.00.

Why it matters

- **Rollback fails** just when it's needed most.
- **Recovery needs hand-written data fixes,** under pressure.
- **Silent failures corrupt data** for days before anyone notices.
- **Teams stop trusting rollback** altogether.

What teams need, before merge

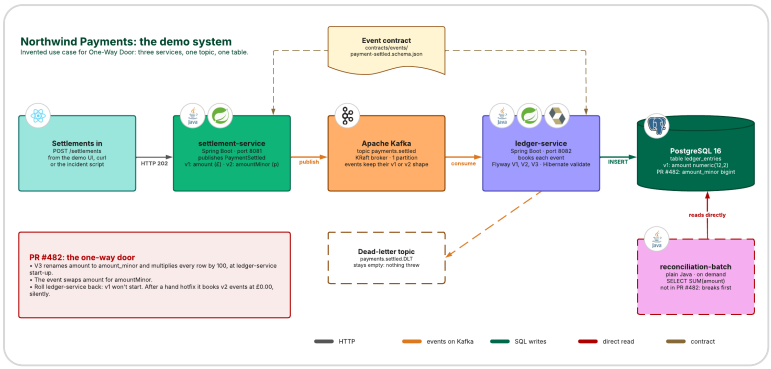
- Is the change **two-way or one-way**?
- **When does the door close,** and what breaks then?
- **Who else reads** the data it changes?
- **How can it be split** so each step can be undone?

Same goal, two kinds of door

Goal	One-way: in one release	Two-way: in steps
Rename a column	**rename it in place**	**add the new column, copy, switch readers, then drop**
Change units	**convert every row in place**	**write both units; switch readers; drop the old**
Replace an event field	**remove the old field, add the new one**	**send both; move every consumer; then stop the old**
Change a cache format	**write only the new format**	**read both formats first; write the new one later**

THE PROBLEM, MADE REAL

Northwind Payments: a sensible refactor, then £0.00



The system. The writer (settlement-service), the reader (ledger-service) and finance's job nobody remembered (reconciliation-batch). Real Java 21 services.



A double can't hold 0.10 exactly. Let's store whole pence, like the card schemes do.

Why PR #482 swaps double for long

```
0.1 + 0.2 == 0.30000000000000004
// before double amount = 268.47;
// after  long amountMinor = 26847;
```

Summed over millions of settlements, totals drift by pennies. A `long` in pence is exact. **The idea is right; shipping it in one deploy isn't.**

One PR, four changes at once (tests green, approved by Tom)

1. **Flyway V3** renames `amount` to `amount_minor`.
2. It **rewrites every row** in pence: £268.47 → 26847.
3. The event swaps `amount` for `amountMinor`.
4. **Not in the PR:** finance's `SUM(amount)` job.

Friday at Northwind, 16:30 to 17:26

Time	Who	What happens	What everyone sees
16:30	 **Priya**	v1 runs: settlements booked in pounds.	`10 rows, all correct`
16:40	 **Tom**	PR #482 ships. V3 runs in 5 ms; settlement-service sends `amountMinor`.	`tests were green`
16:55	 **Grace**	reconciliation-batch, not in the PR, still runs `SUM(amount)`.	`column "amount" does not exist`
16:58	 **Sam**	Sam rolls back **ledger-service only**. The old code won't start.	`missing column [amount]`
17:00	 **Sam**	settlement-service is still v2. Fifty `amountMinor` events queue on Kafka.	`backlog: 50 events`
17:25	 **Dev**	A hand-written hotfix reverses V3. v1 starts and drains the queue.	`ledger-service UP`
17:26	 **Grace**	Fifty settlements booked at **£0.00**. No exception, empty DLT, all green.	`£12,708.11 missing`

Why £0.00 and not an error

```
record PaymentSettled(…,
  double amount, …)   ← missing: 0.0
// "amountMinor": 26847 → ignored
```

The old ledger reads by field name. Jackson ignores the unknown field; the missing `double` stays 0.0. Nothing throws, so nothing reaches the dead-letter topic.

What went wrong

Everyone watched the ledger. **Nobody thought about settlement-service:** it stayed on v2, and the events already on Kafka can't be rolled back. Nor about **reconciliation-batch**, which reads the table but isn't in the PR.



THE SOLUTION

One-Way Door: a GitHub Copilot skill for rollback safety

One folder, `.github/skills/one-way-door`. Copilot loads it when someone reviews a PR, checks a release, changes a migration or an event, or asks "can we roll back?". **Scripts check the mechanical facts; Copilot does the judgement.**

How it works

1 · The PR goes up PR #482 against main › **2 · Someone asks** "can we roll back?" › **3 · The skill loads** its description matches › **4 · Scripts check** migrations and schemas, offline › **5 · Copilot judges** readers, old code vs new state › **6 · The verdict** as a PR comment

GitHub Copilot commented on PR #482 · used `.github/skills/one-way-door`

One-way door check

Verdict: ONE-WAY · Safe rollback window: **none**, closes when V3 runs at ledger-service start-up.

1. **Column renamed and converted in place.** If rolled back, v1 fails its schema check.
2. `amount` **replaced by** `amountMinor`**.** Rollback doesn't remove events already on the topic.
3. **A missing** `amount` **silently becomes £0.00.** Nothing reaches the DLT.

Blast radius: reader	Uses	Breaks when
reconciliation-batch (not in this PR)	`SUM(amount)`	V3 runs
ledger-service	`amount_minor`	rollback
ledger-service consumer	`.amount`	rollback, with a v2 backlog

Suggested split: expand → migrate → contract → drop. scan-migrations: 3 findings · schema-compat: 5 findings.

It predicted every failure on Friday



PREDICTED · seen at 16:58

The old ledger won't start

```
missing column [amount]
```



PREDICTED · seen at 16:55

Finance's job breaks, though it isn't in the PR

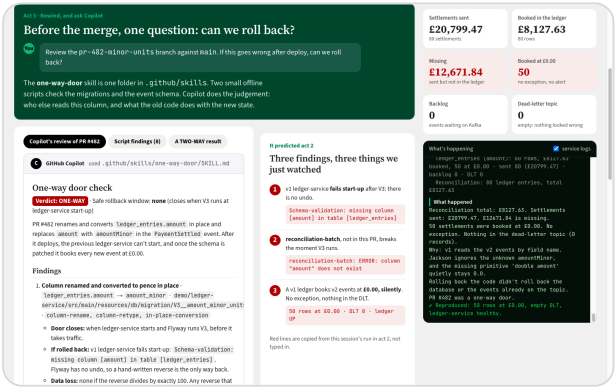
```
column "amount" does not exist
```



PREDICTED · seen at 17:26

Settlements booked at £0.00, silently

```
50 rows at £0.00 · DLT 0 · UP
```



Act 3 of the live demo: each finding beside the real error from the run.

What's in the folder

- `SKILL.md`: the instructions.
- `patterns.md`: 24 one-way patterns, each with its safe split.
- `scan-migrations.mjs`, `schema-compat.mjs`: offline checks, JSON out.
- A path rule, a hook guarding shipped migrations, a plugin.

Where it runs

- **VS Code** agent mode, before you push.
- **Copilot CLI**, or `/one-way-door`.
- **Cloud agent** on the PR on GitHub.
- **Code review**, which can use repository skills.

Guardrails

- Reviews only; never changes code unasked.
- No servers, accounts or network calls.
- The hook fails open: it never blocks your work.
- Readers it can't see are assumed to exist.

THE SOLUTION, PROVEN

Same destination. Four deploys. Each one reversible.

The split ends exactly where PR #482 does, but keeps old and new side by side until nobody needs the old one. This time finance's job moves with the ledger.

1 Expand

Table: add nullable `amount_minor`; write both.

Event: send both fields.

Finance: unchanged, `SUM(amount)`.

⇄ old code ignores the new column

2 Migrate

Table: backfill; check every row agrees.

Event: still both.

Finance: also flags any mismatch.

⇄ both columns correct

3 Contract

Table: read only `amount_minor`; a trigger fills `amount`.

Event: still both; no amount goes to the DLT.

Finance: switches to `SUM(amount_minor)`.

⇄ `amount` kept up to date

4 Drop

Table: drop `amount` and the trigger.

Event: `amountMinor` only.

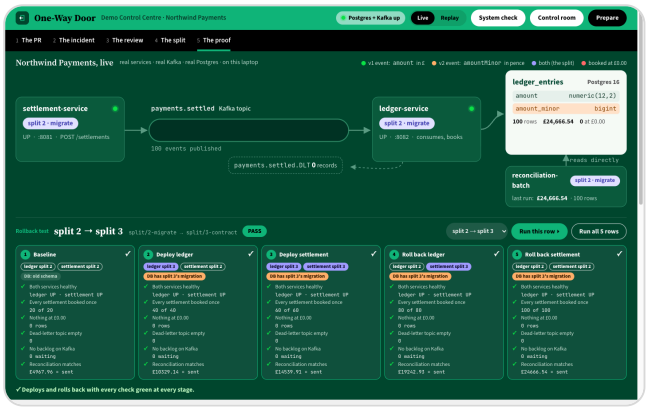
Finance: already moved in step 3.

⇄ nothing uses the old any more

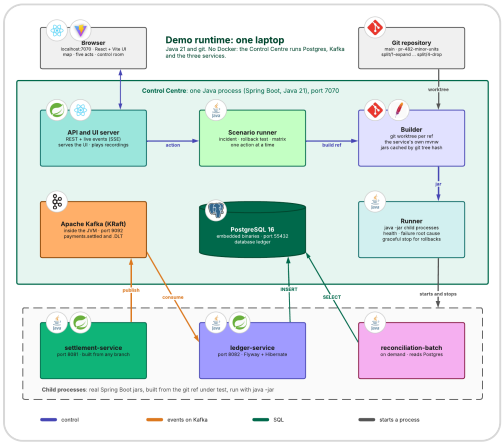
The proof: the rollback matrix, run for real

old → new	1 baseline	2 deploy ledger	3 deploy settle-ment	4 roll back ledger	5 roll back settlement	result
`main → split 1`	✓ 20/20	✓ 40/40	✓ 60/60	✓ 80/80	✓ 100/100	PASS
`split 1 → split 2`	✓ 20/20	✓ 40/40	✓ 60/60	✓ 80/80	✓ 100/100	PASS
`split 2 → split 3`	✓ 20/20	✓ 40/40	✓ 60/60	✓ 80/80	✓ 100/100	PASS
`split 3 → split 4`	✓ 20/20	✓ 40/40	✓ 60/60	✓ 80/80	✓ 100/100	PASS
`main → PR #482`	✓ 20/20	✗ 20 at £0	✗ 20 at £0	✗ 20 at £0	✗ 20 at £0	FAIL

After every stage: 20 settlements, then six checks. Both services healthy, each booked once, nothing at £0.00, empty DLT, no backlog, reconciliation equals money sent. **PR #482 fails at stage 2, before anyone presses rollback.**



Act 5, live: split 2 → split 3 deploys and rolls back in five stages; every check green in about 20 seconds.



The demo runtime: one laptop, Java 21 and git, no Docker.

The live demo, in five acts

A Java Control Centre starts real Postgres and Kafka, builds each service from git and runs it. The browser shows **the PR**, **the incident**, **the review**, **the split** and **the proof**. Every rollback runs the previous commit's code against today's data. Windows and macOS; a recorded run is the fallback.

Next: readers in other repositories

Search related repositories at their deployed versions, and **assume any repository it can't reach reads the data**. The verdict states its coverage, and the split names each repository's owner and order.

Every merge is a door. Know which kind before you walk through it.

```
cp -R .github/skills/one-way-door your-repo/.github/skills/
```